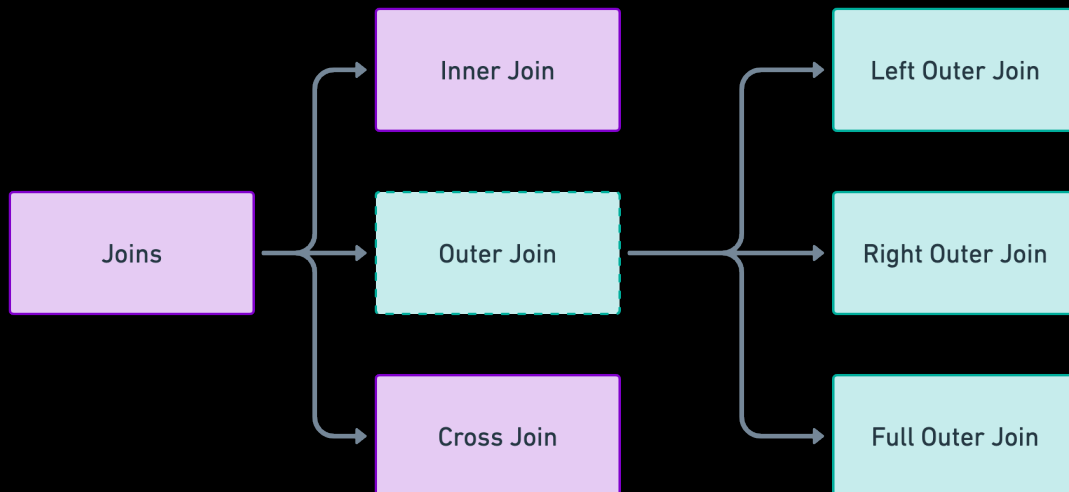


Lab 4 - Joins and Set Operations

Krishanu Shah, Amod Nijasure



Inner Join

Returns only those records that have matching values [data that appears in both tables]

```
SELECT <columns>
FROM table1 INNER JOIN table2
ON table1.matching_column = table2.matching_column;
```

// INNER JOIN is the default JOIN in MySQL - t1 JOIN t2 performs an INNER JOIN

```
mysql> select * from t1;
+-----+-----+
| t1_id | attr_1 | attr_2 |
+-----+-----+
| 1 | A | P |
| 2 | B | Q |
| 3 | C | R |
| 4 | D | S |
| 5 | E | T |
+-----+-----+
```

```
mysql> select * from t2;
+-----+-----+
| t2_id | attr_a | attr_b |
+-----+-----+
| 1 | A | V |
| 2 | B | W |
| 5 | C | X |
| 6 | F | Y |
| 7 | G | Z |
+-----+-----+
```

```
mysql> select * from t1 inner join t2 on t1.t1_id = t2.t2_id;
+-----+-----+-----+-----+-----+-----+
| t1_id | attr_1 | attr_2 | t2_id | attr_a | attr_b |
+-----+-----+-----+-----+-----+-----+
| 1 | A | P | 1 | A | V |
| 2 | B | Q | 2 | B | W |
| 5 | E | T | 5 | C | X |
+-----+-----+-----+-----+-----+-----+
```

```
mysql> select * from t1, t2 where t1.t1_id = t2.t2_id;
+-----+-----+-----+-----+-----+-----+
| t1_id | attr_1 | attr_2 | t2_id | attr_a | attr_b |
+-----+-----+-----+-----+-----+-----+
| 1 | A | P | 1 | A | V |
| 2 | B | Q | 2 | B | W |
| 5 | E | T | 5 | C | X |
+-----+-----+-----+-----+-----+-----+
```

Similar result achieved with cross join + where to match common columns
(a.k.a. EQUI JOIN)

Left (Outer) Join

Returns all the rows of the table on the left side of the join [Table 1] and matches rows for the table on the right side of the join [Table 2].

For the rows for which there is no matching row on the **RIGHT** side, the result-set will contain NULL.

```
SELECT <columns>
FROM table1 LEFT JOIN table2
ON table1.matching_column = table2.matching_column;

// can use LEFT OUTER JOIN and LEFT JOIN interchangeably
```

Right (Outer) Join

Similar to LEFT JOIN, The RIGHT OUTER JOIN selects data from the right table [Table 2] and matches this data with the rows from the left table [Table 1].

For the rows for which there is no matching row on the **LEFT** side, the result-set will contain NULL.

```
SELECT <columns>
FROM table1 RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;

// can use RIGHT OUTER JOIN and RIGHT JOIN interchangeably
```

```
mysql> select * from t1;
+-----+-----+-----+
| t1_id | attr_1 | attr_2 |
+-----+-----+-----+
| 1 | A | P |
| 2 | B | Q |
| 3 | C | R |
| 4 | D | S |
| 5 | E | T |
+-----+-----+-----+
```

```
mysql> select * from t1 left join t2 on t1.t1_id = t2.t2_id;
+-----+-----+-----+-----+-----+-----+
| t1_id | attr_1 | attr_2 | t2_id | attr_a | attr_b |
+-----+-----+-----+-----+-----+-----+
| 1 | A | P | 1 | A | V |
| 2 | B | Q | 2 | B | W |
| 3 | C | R | NULL | NULL | NULL |
| 4 | D | S | NULL | NULL | NULL |
| 5 | E | T | 5 | C | X |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Left join - NULL values on right side of return set

```
mysql> select * from t2;
+-----+-----+-----+
| t2_id | attr_a | attr_b |
+-----+-----+-----+
| 1 | A | V |
| 2 | B | W |
| 5 | C | X |
| 6 | F | Y |
| 7 | G | Z |
+-----+-----+-----+
```

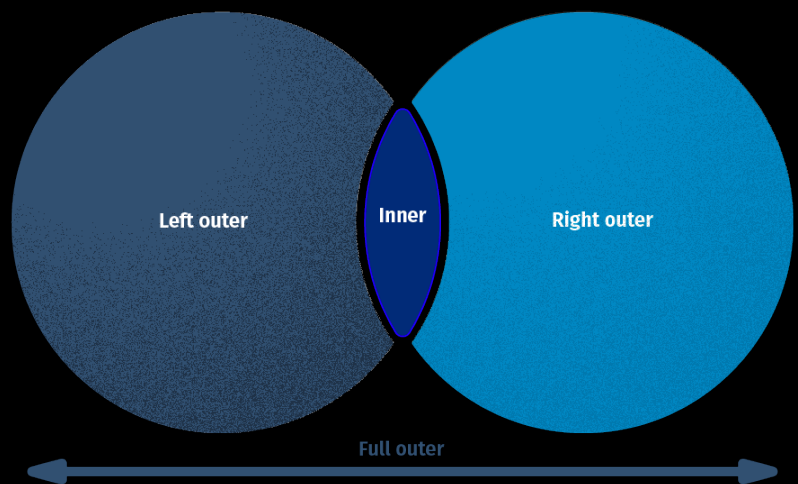
```
mysql> select * from t1 right join t2 on t1.t1_id = t2.t2_id;
+-----+-----+-----+-----+-----+-----+
| t1_id | attr_1 | attr_2 | t2_id | attr_a | attr_b |
+-----+-----+-----+-----+-----+-----+
| 1 | A | P | 1 | A | V |
| 2 | B | Q | 2 | B | W |
| 5 | E | T | 5 | C | X |
| NULL | NULL | NULL | 6 | F | Y |
| NULL | NULL | NULL | 7 | G | Z |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Right join - NULL values on left side of return set

Full (Outer) Join

The FULL OUTER JOIN returns a result that includes rows from both left and right tables.

In case, no matching rows exist for the row in the left table, the columns of the right table will have nulls and vice versa.



```
SELECT <columns>
FROM table1 FULL JOIN table2
ON table1.matching_column = table2.matching_column;
```

IMPORTANT - Some versions of MySQL don't support FULL JOIN, and this error is returned -

```
select * from t1 full outer join t2 on t1.t1_id = t2.t2_id;
```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'outer join t2 on t1.t1_id = t2.t2_id' at line 1

```
select * from t1 full join t2 on t1.t1_id = t2.t2_id;
```

ERROR 1054 (42S22): Unknown column 't1.t1_id' in 'on clause'

Workaround

```
SELECT <columns>
FROM table1 LEFT JOIN table2
ON table1.matching_column = table2.matching_column;
UNION
SELECT <columns>
FROM table1 RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```

```
mysql> select * from t1;
+-----+-----+-----+
| t1_id | attr_1 | attr_2 |
+-----+-----+-----+
| 1 | A | P |
| 2 | B | Q |
| 3 | C | R |
| 4 | D | S |
| 5 | E | T |
+-----+-----+-----+

mysql> select * from t2;
+-----+-----+-----+
| t2_id | attr_a | attr_b |
+-----+-----+-----+
| 1 | A | V |
| 2 | B | W |
| 5 | C | X |
| 6 | F | Y |
| 7 | G | Z |
+-----+-----+-----+
```

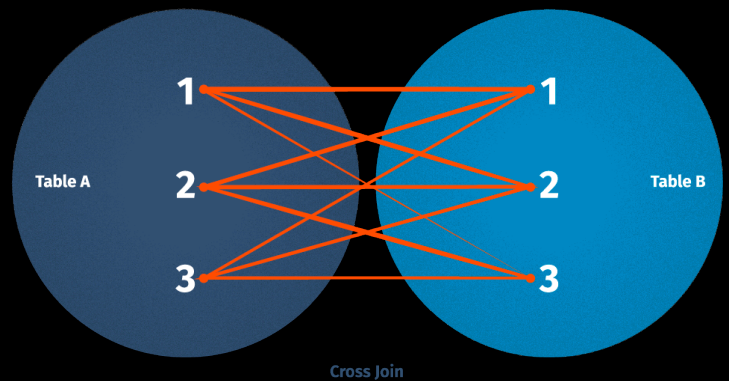
```
mysql> select * from t1 right join t2 on t1.t1_id = t2.t2_id
union select * from t1 left join t2 on t1.t1_id = t2.t2_id;
+-----+-----+-----+-----+-----+-----+
| t1_id | attr_1 | attr_2 | t2_id | attr_a | attr_b |
+-----+-----+-----+-----+-----+-----+
| 1 | A | P | 1 | A | V |
| 2 | B | Q | 2 | B | W |
| 5 | E | T | 5 | C | X |
| NULL | NULL | NULL | 6 | F | Y |
| NULL | NULL | NULL | 7 | G | Z |
| 3 | C | R | NULL | NULL | NULL |
| 4 | D | S | NULL | NULL | NULL |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.01 sec)
```

Cross Join (Cartesian Join)

Returns all combinations of rows from each table.

Number of rows returned
 $\text{rows in t1} \times \text{rows in t2}$

Number of columns returned
 $\text{columns in t1} + \text{columns in t2}$



```
SELECT <columns>
FROM table1 CROSS JOIN table2
Or
SELECT <columns>
FROM table1, table2
```

```
mysql> select * from t1;
```

t1_id	attr_1	attr_2
1	A	P
2	B	Q
3	C	R
4	D	S
5	E	T

```
mysql> select * from t2;
```

t2_id	attr_a	attr_b
1	A	V
2	B	W
5	C	X
6	F	Y
7	G	Z

```
mysql> select * from t1, t2;
```

t1_id	attr_1	attr_2	t2_id	attr_a	attr_b
5	E	T	1	A	V
4	D	S	1	A	V
3	C	R	1	A	V
2	B	Q	1	A	V
1	A	P	1	A	V
5	E	T	2	B	W
4	D	S	2	B	W
3	C	R	2	B	W
2	B	Q	2	B	W
1	A	P	2	B	W
5	E	T	5	C	X
4	D	S	5	C	X
3	C	R	5	C	X
2	B	Q	5	C	X
1	A	P	5	C	X
5	E	T	6	F	Y
4	D	S	6	F	Y
3	C	R	6	F	Y
2	B	Q	6	F	Y
1	A	P	6	F	Y
5	E	T	7	G	Z
4	D	S	7	G	Z
3	C	R	7	G	Z
2	B	Q	7	G	Z
1	A	P	7	G	Z

25 rows in set (0.00 sec)

Additional Logical Operators

ANY vs ALL

Teachers Table

ID	Name	Age
1	Adam	28
2	Vijay	40
3	Noel	33
4	Ishan	30
5	Jamal	50

Students Table

ID	Name	Age
1	Ujwal	21
2	Ram	19
3	Aman	32
4	Quinn	20
5	Tim	18

```
SELECT * FROM  
teachers WHERE age <  
ANY ( SELECT age from  
students );
```

```
SELECT * FROM  
teachers WHERE age >  
ALL ( SELECT age from  
students );
```

rows returned where age < at least 1 value in
(21, 19, 32, 20, 18)

rows returned where age > all values in
(21, 19, 32, 20, 18)

ID	Name	Age
1	Adam	28
4	Ishan	30

ID	Name	Age
2	Vijay	40
3	Noel	33
5	Jamal	50

Exists Operator

The SQL EXISTS operator executes the outer SQL query if the subquery is not an empty result-set.

If the inner query returns anything except an empty-set, the outer query will execute.

```
mysql> select * from orders where exists
(select FALSE);
```

order_id	customer_id	order_date
1	1	2023-01-01
2	3	2023-01-12
3	9	2023-01-12
4	2	2023-01-23
5	5	2023-04-04
6	3	2023-04-25
7	5	2023-05-16
8	6	2023-09-27
9	6	2023-11-08
10	10	2023-12-19

10 rows in set (0.00 sec)

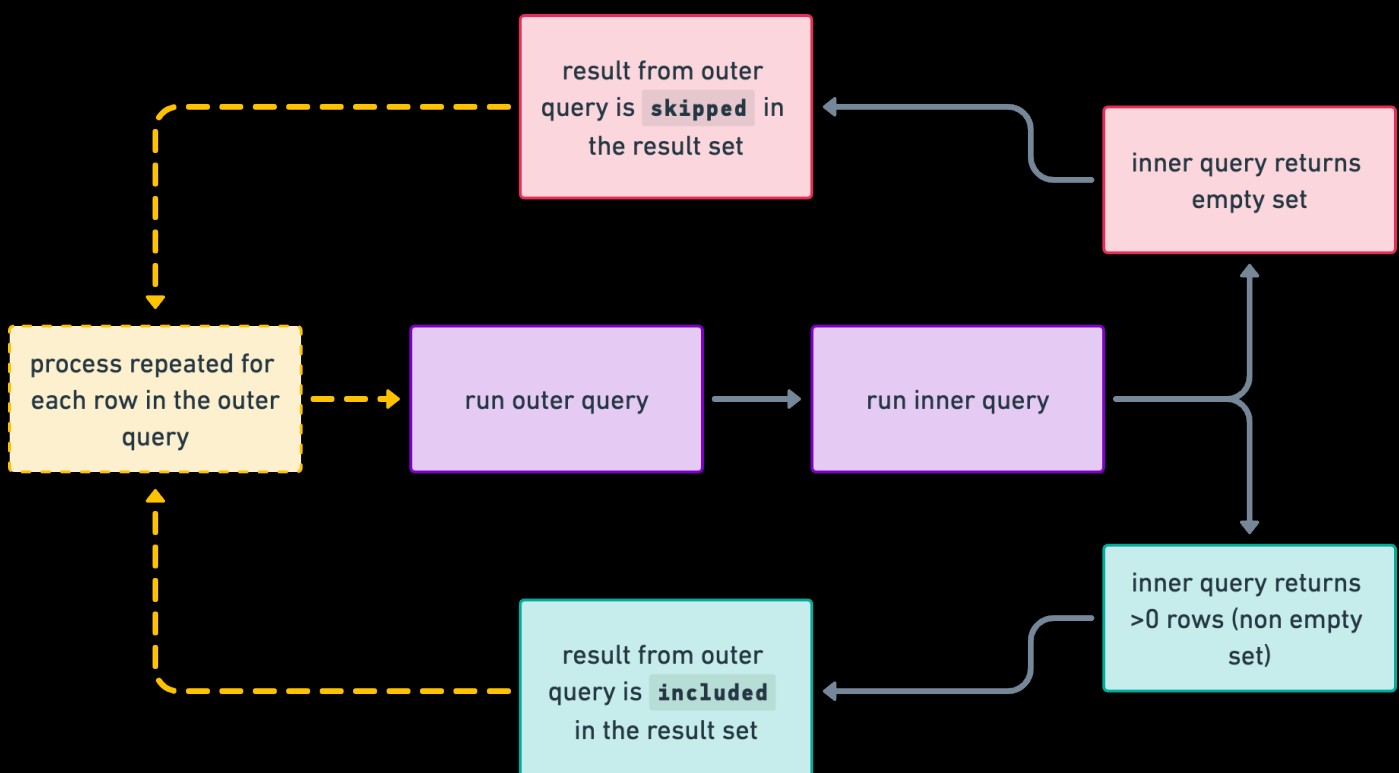
```
mysql> select * from orders where exists
(select NULL);
```

order_id	customer_id	order_date
1	1	2023-01-01
2	3	2023-01-12
3	9	2023-01-12
4	2	2023-01-23
5	5	2023-04-04
6	3	2023-04-25
7	5	2023-05-16
8	6	2023-09-27
9	6	2023-11-08
10	10	2023-12-19

10 rows in set (0.00 sec)

Even if the inner subquery returns NULL or FALSE, the outer query will be executed

// usage: `SELECT .. .. WHERE EXISTS (<SUBQUERY>)`



Working of the EXISTS operator

An example - to clear things up

What happens when the inner subquery is dependant on the outer query?

Customers Table

ID	Name
1	Adam
2	Vijay
3	Noel
4	Ishan

Orders Table

O_ID	C_ID	Item	Quantity
1	2	Rock	21
2	8	Paper	19
3	1	Scissors	32
4	9	Lizard	20
5	4	Spock	18

```
SELECT * FROM
customers WHERE
EXISTS (SELECT * FROM
orders WHERE
orders.C_ID =
customers.ID)
```

ID	Name
1	Adam
2	Vijay
3	Noel
4	Ishan

```
SELECT * FROM
customers WHERE
EXISTS (SELECT *
FROM orders WHERE
orders.C_ID = 1)
```

Inner Query changes
for each row

Subquery
returns >0
rows, condition
matched

Add to result set

ID	Name
1	Adam

ID	Name
1	Adam
2	Vijay
3	Noel
4	Ishan

```
SELECT * FROM
customers WHERE
EXISTS (SELECT *
FROM orders WHERE
orders.C_ID = 2)
```

Subquery
returns >0
rows, condition
matched

Add to result set

ID	Name
1	Adam
2	Vijay

ID	Name
1	Adam
2	Vijay
3	Noel
4	Ishan

```
SELECT * FROM
customers WHERE
EXISTS (SELECT *
FROM orders WHERE
orders.C_ID = 3)
```

Subquery
returns empty
set, condition
not met

Not Added

ID	Name
1	Adam
2	Vijay

ID	Name
1	Adam
2	Vijay
3	Noel
4	Ishan

```
SELECT * FROM
customers WHERE
EXISTS (SELECT *
FROM orders WHERE
orders.C_ID = 4)
```

Subquery
returns >0
rows, condition
matched

Add to result set

Result Set	
ID	Name
1	Adam
2	Vijay
4	Ishan